

SIMATS ENGINEERING
ASSESSMENT TOOL II – REVERSE ENGINEERING TASK (MEDIUM)

Course Name: Ethical Hacking for Secure Networks	Code: ITA1404	CO: CO2
Duration: 90 min Date:	Total Marks:	Reg. No / Name : 192311053

CO2: Execute footprinting in social engineering such as DNS analysis, competitive intelligence gathering, and attack simulations to collect relevant information for identifying potential security risks.(BL3)
Assessment Tool Weightage: 30%

Instructions: Answer all questions. Each question carries 10 marks.

Task: 1 A cybersecurity team discovers a suspicious executable named **FootPrintX.exe** on a penetration tester's workstation. The application appears to collect information about target organizations, including domain names, IP addresses, and network details. The source code is unavailable, and the analyst must reverse engineer the binary to understand its functionality. Your task is to identify how the tool performs footprinting activities.

Questions:

1. Which functions or modules are responsible for gathering target information?
2. What footprinting techniques are implemented in the executable?
3. How can the information collection process be disabled or modified?

Task: 2 A file named **DNSScan.exe** has been found during a security audit. Preliminary analysis suggests that the program attempts to gather DNS information from external servers. Security analysts suspect that it performs DNS zone transfer operations to obtain detailed records of a target domain. Reverse engineer the executable to determine its behavior and associated risks.

Questions:

1. Which function is responsible for initiating DNS requests?
2. Does the application attempt an AXFR (Zone Transfer) request? Justify your answer.
3. What information can an attacker obtain using the extracted DNS records?

Task: 3 An organization identifies a tool called **IntelGather.exe** that collects publicly available information from websites and online repositories. The application is suspected of gathering employee details, email addresses, and technology information related to organizations. Since the source code is unavailable, reverse engineering is required to determine the data collection mechanisms. Analyze the executable and identify its intelligence-gathering capabilities.

Questions:

1. What types of information are collected by the application?
2. Which external websites or services are accessed during execution?
3. How can the collected intelligence be misused by attackers?

Task: 4 A suspicious program named **MailCraft.exe** is discovered on a compromised system. During execution, the application automatically creates convincing email messages that imitate legitimate organizations. Analysts suspect that the tool is used to perform phishing attacks against employees. Reverse engineer the executable to identify how the emails are generated and customized.

Questions:

1. Which functions are responsible for generating phishing emails?
2. What information is used to personalize the messages?
3. How can the phishing functionality be prevented or removed?

Task: 5 A binary file named **PortalClone.exe** imitates the login page of a popular online service. Users who enter their credentials unknowingly submit their information to the application. The security team needs to understand how the credentials are captured, stored, and transmitted. Reverse engineer the program to identify its malicious behavior.

Questions:

1. How does the application capture usernames and passwords?
2. Where are the captured credentials stored or transmitted?
3. What security risks arise from the use of such applications?

Code of Conduct:

I S Venkata Abhishek(192311053)(Name / Reg. No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:Abhishek

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of System	20%				
Decomposition	25%				
Identification of Components	20%				
Reconstruction	20%				
Documentation	15%				

Criteria	Understanding of System	Decomposition	Identification of Components	Reconstruction	Documentation	Total
Q1						
Q2						
Q3						
Q4						
Q5						

Task 1 — FootPrintX.exe

1. Which functions or modules are responsible for gathering target information?

During reverse engineering, the analyst should identify functions that perform **DNS lookups, WHOIS queries, IP discovery, network enumeration, and HTTP/HTTPS communication**.

Typical indicators include:

1. DNS resolution functions such as `getaddrinfo()` or `gethostbyname()`.
2. Network communication functions such as `connect()`, `send()`, and `recv()`.
3. Windows networking APIs such as WinINet or WinHTTP functions.
4. Functions that construct URLs for WHOIS or search-engine queries.
5. Functions that parse returned domain, IP, and network information.
6. Modules responsible for storing the collected information in files, memory, or databases.

Strings such as `whois`, `dns`, `domain`, `ip`, `http`, `https`, and API URLs can also help identify the relevant modules.

Conclusion: The information-gathering component is normally identified by tracing network-related APIs and the functions that process their responses.

2. What footprinting techniques are implemented in the executable?

Based on the scenario, FootPrintX.exe may implement several reconnaissance techniques:

Domain enumeration — identifying target domains and subdomains.

DNS reconnaissance — resolving domain names and collecting DNS information.

IP address discovery — mapping domains or hosts to IP addresses.

WHOIS lookup — obtaining publicly available registration information.

Network information gathering — collecting information about hosts and network ranges.

Web-based reconnaissance — querying public websites or APIs.

Information aggregation — combining the collected information into a target profile.

The analyst can confirm these techniques by examining imported functions, embedded strings, network traffic, and the program's control flow.

Conclusion: The executable appears to automate passive and potentially active reconnaissance techniques to build an attack-surface profile.

3. How can the information collection process be disabled or modified?

The collection process can be disabled or modified during analysis by:

- Identifying the function responsible for initiating network requests.
- Removing or bypassing calls to that function in a controlled analysis copy.
- Modifying configuration values that specify target domains or collection settings.
- Blocking the executable's network communication using firewall rules.
- Disabling specific collection modules if the application supports modular configuration.
- Patching the binary so that the relevant collection routine is not executed.
- Running the executable in an isolated sandbox to prevent unintended communication.

For defensive analysis, the safest approach is to **isolate the executable and block network access** rather than modifying a potentially malicious binary on a production system.

Conclusion: Network controls and disabling the relevant collection routines can prevent the executable from gathering target information.

Task 2 — DNSScan.exe

1. Which function is responsible for initiating DNS requests?

Without the actual binary, the exact function name cannot be stated. During reverse engineering, analysts should look for DNS-related APIs and libraries.

Common Windows indicators include:

DnsQuery()

DnsQueryEx()

getaddrinfo()

gethostbyname()

The analyst can identify the responsible function by examining:

Imported functions.

Cross-references to DNS APIs.

Parameters passed to the APIs.

Domain names supplied to the functions.

Network traffic generated during execution.

If the executable constructs a DNS query manually, the analyst should trace the function that creates and sends the DNS packet.

Conclusion: The responsible function is the function that invokes the DNS resolver/API or constructs and sends the DNS request.

2. Does the application attempt an AXFR (Zone Transfer) request? Justify your answer.

The actual binary is required for a definitive answer. An AXFR attempt can be confirmed by observing one or more of the following:

The string AXFR in the executable.

DNS query packets containing the **AXFR query type**.

Code that constructs a DNS query with the AXFR record type.

Network traffic directed to an authoritative DNS server on port 53.

Functions specifically implementing DNS zone-transfer logic.

If packet capture shows an AXFR request followed by DNS zone data, the application is attempting a zone transfer.

If only ordinary A, AAAA, MX, NS, or TXT queries are observed, there is insufficient evidence that the application performs AXFR.

Conclusion: AXFR behavior must be established through binary analysis or network observation rather than assumed from the filename.

3. What information can an attacker obtain using the extracted DNS records?

If an unauthorized zone transfer succeeds, an attacker may obtain:

- Complete or extensive lists of hostnames.
- Subdomains.
- IP addresses.
- Mail-server information.
- Name-server information.
- CNAME relationships.
- Internal naming conventions.
- Development and testing hosts.
- Other DNS records contained in the zone.

This information can help attackers map an organization's external infrastructure and identify systems for further security testing.

Conclusion: Unauthorized DNS zone information can significantly reduce the effort required for reconnaissance and attack-surface mapping.

Task 3 — IntelGather.exe

1. What types of information are collected by the application?

Based on the scenario, IntelGather.exe may collect publicly available intelligence such as:

Employee names.

1. Email addresses.
2. Job titles and organizational roles.
3. Company and domain names.
4. Subdomains and IP addresses.

5. Publicly available technology information.
6. Website information.
7. Public documents.
8. Social-media or professional-profile information.
9. Information from online repositories.
10. Publicly exposed technical details.

The analyst can confirm these capabilities by examining strings, URLs, API endpoints, imported libraries, and network traffic.

Conclusion: The application appears to perform OSINT collection to construct a detailed profile of an organization and its employees.

2. Which external websites or services are accessed during execution?

The exact websites cannot be identified without the executable. During reverse engineering, analysts should examine:

- Embedded URLs and domain names.
- HTTP/HTTPS requests.
- DNS queries generated by the application.
- API endpoints.
- Browser or web-request libraries.
- Configuration files containing external services.
- Network captures from a controlled sandbox.

Possible categories of services include:

- Search engines.
- WHOIS/RDAP services.
- DNS services.
- Public company websites.
- Professional/social networking sites.
- Public code repositories.
- Technology-profiling services.
- Public APIs.

Conclusion: The exact external services should be determined from the executable's strings, imports, configuration, and observed network traffic rather than assumed.

3. How can the collected intelligence be misused by attackers?

Collected intelligence can be misused for:

Phishing attacks — using employee names and roles to create convincing messages.

Spear phishing — targeting specific employees with personalized content.

Social engineering — exploiting information about employees and organizational structure.

Credential attacks — identifying usernames or email formats.

Infrastructure reconnaissance — identifying potentially vulnerable systems.

Business-email compromise — impersonating employees or executives.

Technology-specific attacks — identifying technologies that may contain vulnerabilities.

Target prioritization — identifying high-value employees and systems.

Conclusion: Even publicly available information can become dangerous when aggregated because it provides attackers with a detailed picture of the organization's people, technology, and infrastructure.

Task 4 — MailCraft.exe

1. Which functions are responsible for generating phishing emails?

The exact functions require the binary for identification. During reverse engineering, analysts should look for functions related to:

- Email/message creation.
- String formatting and template processing.
- SMTP communication.
- HTTP/HTTPS requests to an email service.
- Recipient-list processing.
- Attachment generation.
- HTML email construction.

Relevant indicators may include strings such as:

SMTP, Subject, From, To, Body, HTML, attachment, or email-related URLs.

The analyst should trace the program from its input data through the message-generation function and finally to the network-sending function.

Conclusion: The phishing-email functionality consists of message-template generation, personalization, and email transmission components.

2. What information is used to personalize the messages?

The program may use information such as:

- Recipient's name.
- Employee role or job title.
- Organization name.
- Department.
- Email address.
- Company branding.
- Previously collected organizational information.
- Personalized subject lines.
- Links or other content designed to appear legitimate.

For example, a message may address an employee by name and refer to their organization or department, making the phishing message more convincing.

Conclusion: Personalization increases the credibility of phishing messages and therefore increases the likelihood that recipients will interact with them.

3. How can the phishing functionality be prevented or removed?

Organizations can prevent phishing functionality through multiple controls:

- Remove or quarantine the malicious executable.
- Block its network communication.
- Detect and block known malicious hashes and indicators.
- Use endpoint detection and response (EDR).
- Restrict unauthorized software execution.
- Implement strong email filtering.
- Use SPF, DKIM, and DMARC to reduce email spoofing.
- Train employees to recognize phishing.
- Use multi-factor authentication (MFA).
- Monitor suspicious outbound email activity.
- Patch and secure compromised systems.
- Reset credentials if compromise is suspected.

If analyzing the executable itself, the organization should preserve evidence and use an isolated environment rather than simply executing or modifying it on a production workstation.

Conclusion: A combination of endpoint protection, email security, authentication controls, monitoring, and user awareness is required to prevent phishing abuse.

Task 5 — PortalClone.exe

1. How does the application capture usernames and passwords?

A credential-phishing application commonly presents a **fake login interface** that resembles a legitimate service.

The application may capture credentials when the victim submits the form. During reverse engineering, analysts should look for:

1. Input-field handling functions.
2. Username/password variables.
3. Form-submission handlers.
4. String or buffer operations involving credentials.
5. HTTP/HTTPS POST requests.
6. Functions that write captured data to a file or database.
7. Network functions that transmit submitted data.

The analyst should trace the data flow:

User input → credential variables → storage/processing → transmission

This helps establish exactly how credentials are captured.

Conclusion: PortalClone.exe captures credentials by intercepting the values entered into its imitation login form and processing them for storage or transmission.

2. Where are the captured credentials stored or transmitted?

The exact destination cannot be determined without the binary. Reverse engineering can identify the destination by examining:

- File-writing functions.
- Database operations.
- Registry access.
- Temporary files.
- Embedded URLs/IP addresses.
- HTTP/HTTPS requests.
- Network sockets.
- Command-and-control or collection endpoints.

Possible destinations include:

- A local file.
- A database.
- Application storage.
- A remote web server.
- An attacker-controlled server.

Network monitoring in an isolated environment can reveal whether credentials leave the system.

Conclusion: Analysts should trace credential data from the input point to its final storage or network destination to determine the complete exfiltration path.

3. What security risks arise from the use of such applications?

Credential-phishing applications create serious security risks, including:

1. **Credential theft** — usernames and passwords can be stolen.
2. **Account takeover** — attackers can use stolen credentials to access accounts.
3. **Identity theft** — compromised accounts can expose personal information.
4. **Data breaches** — stolen credentials may provide access to organizational data.
5. **Lateral movement** — compromised accounts can help attackers access additional systems.
6. **Financial loss** — compromised accounts may be used for fraudulent activities.
7. **Privacy violations** — personal and confidential information may be exposed.
8. **Reputation damage** — organizations can suffer loss of customer trust.
9. **Password reuse attacks** — stolen credentials may work on other services.
10. **Business disruption** — compromised accounts can be used to disrupt operations.

Conclusion: Fake-login applications are dangerous because they directly target user credentials. MFA, password managers, phishing-resistant authentication, secure email filtering, endpoint protection, and user awareness can significantly reduce the risk.